

# PRESENTACION DEL CÓDIGO

Explicación del código realizado para la prueba  
técnica

Arbey Giovanny Chica Gil

Medellín, Antioquia, Colombia

2025

1: Archivo inicial del backend, hecho con express

- en el código se ven las importaciones básicas para el funcionamiento

- inicialización del servicio, con express

- se observa la inicialización de especificaciones para el funcionamiento, como:

“Morgan” para entender las solicitudes http,

“Express.json” para que el servidor entienda el formato JSON que va a recibir

“COR2 para permitir el consumo de la api desde el frontend, permitiendo la salida y entrada de información desde una url específica

“ruta” inicializa el comportamiento de el servicio en como se va a manejar y las rutas permitidas por la api

“port” se establece el puerto de salida de la api, mediante el cual va permitir la entrada y salida de la información

“initializeDatabase” el método que inicializa la base de datos, en caso de que no exista, la crea, así como las tablas respectivas

“listen” permite que el servidor esté listo para que empiece a trabajar

por último tiene un console.log que permite ver por una consola el estado del servidor, si está en ejecución, o si ocurrió algún error

```
1 import Express from "express"
2 import cors from "cors"
3 import morgan from "morgan"
4
5 // inicializar express
6 const app = Express()
7
8 // middleware
9 app.use(morgan('dev')) // para que express pueda entender las peticiones
10 app.use(Express.json()) // para que express pueda entender json
11 app.use(Express.urlencoded({ extended: true })) // para que express pueda entender formularios
12
13 // Configura CORS para permitir cookies desde el frontend
14 app.use(cors({
15   origin: 'http://localhost:5173', // url FrontEnd (react + vite)
16   credentials: true
17 }));
18
19 // rutas
20 import ruta from "../src/routes.js"
21 app.use('/api', ruta)
22 // condiciones de la ruta equivocada -> dar info de cual es la ruta correcta
23 app.use((req, res) => {
24   res.status(404).json({ message: 'Ruta no encontrada, la ruta a la que debe ingresar es: http://localhost:3000/api' })
25 })
26
27 // definir puerto
28 const PORT = 3000
29 app.set('port', PORT)
30
31 // conexión base de datos
32 import { initializeDatabase } from '../src/database/conexion.js'
33 initializeDatabase()
34
35 // configuración del puerto y servidor
36 app.listen(app.get('port'))
37
38 // mensaje de inicio
39 console.log(`\x1b[36mServidor corriendo en el puerto ${app.get('port')} y servidor local http://localhost:${app.get('port')}`)
40
41 // exportar app
42 export default app
```

Método de conexión con la base de datos, estableciendo los parámetros necesarios como:  
- el motor, - la ubicación, el nombre, - los logs de “verificación”

```
1  /**
2   * Configuración de La base de datos
3   * Conexión a SQLite
4   * Crear base de datos
5   */
6  const sequelize = new Sequelize({
7    dialect: 'sqlite',
8    storage: './database.sqlite', // Ruta del archivo de La base de datos
9    logging: false, // Opcional: desactiva Logs de SQL
10 });
11
```

“initializeDatabase”

método que permite la conexión con la base de datos “escuchando” si existe, si no la crea, y así mismo con las tablas que debe tener

```
1  /**
2   * Función para conectar y sincronizar modelos
3   * Asegura que todos Los modelos estén registrados y sincronizados con La base de datos
4   */
5  export async function initializeDatabase() {
6    try {
7      // Importar todos Los modelos aquí para asegurar que se registren
8      await import('../models/Tareas.model.js');
9
10     await sequelize.authenticate();
11     console.log('Conexión a la base de datos exitosa');
12
13     await sequelize.sync({ alter: true });
14     console.log('Sincronización de modelos exitosa - Tablas creadas/actualizadas');
15   } catch (error) {
16     console.error('Error al conectar o sincronizar:', error);
17   }
18 }
19
```

Modelo de la tabla “Tareas” en la base de datos, utilizando el ORM Sequelize se establece el nombre de la tabla, los campos y sus propiedad para la creación de la tabla en la base de datos y su funcionamiento en el sistema

```
1  /**
2   * Modelo para Las tareas
3   * ORM -> Sequelize
4   * para la tabla "tareas" en la base de datos
5   */
6  const Tareas = sequelize.define('Tareas', {
7    id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
8
9    title: { type: DataTypes.STRING, allowNull: false },
10
11    description: { type: DataTypes.TEXT, allowNull: false },
12
13    status: { type: DataTypes.ENUM('pendiente', 'en_progreso', 'completada'), defaultValue: 'pendiente' },
14
15    priority: { type: DataTypes.ENUM('1', '2', '3'), defaultValue: '3' },
16
17    due_date: { type: DataTypes.DATE, allowNull: false }
18  }, {
19    tableName: 'tareas', timestamps: true
20  });
21
22  export default Tareas;
```

modelo de la tabla “Tareas” para usar en las validaciones del sistema y los “requisitos” mínimos para la aceptación y funcionamiento de los procesos/métodos de interacción con la base de datos

```
1  /**
2   * Esquema de validación para Las tareas
3   * Utiliza Joi para definir las reglas de validación
4   */
5  const tareaSchema = Joi.object({
6    title: Joi.string().min(3).max(100).required(),
7    description: Joi.string().min(5).max(500).required(),
8    status: Joi.string().valid('pendiente', 'en_progreso', 'completada').default('pendiente'),
9    priority: Joi.string().valid('1', '2', '3').default('3'),
10    due_date: Joi.date().greater('now').required()
11  });
12
13  export default tareaSchema;
```

Configuración de las rutas mediante se va a manejar la información con las solicitudes/peticiones HTTP en el sistema para la interacción con la información almacenada o por almacenar, llamando a los respectivos método según la ruta

```
1 // inicar express
2 const ruta = Express()
3
4 /**
5  * Rutas para la gestión de tareas
6  * con todos los metodos http necesarios y sus funciones
7  */
8 ruta.post('/tasks', crearTarea)
9 ruta.get('/tasks', obtenerTareas)
10 ruta.get('/tasks/:id', obtenerTareaPorId)
11 ruta.put('/tasks/:id', actualizarTarea)
12 ruta.delete('/tasks/:id', eliminarTarea)
13 // url por defecto
14 ruta.get('/', (req, res) => {
15   res.send(
16     'Bienvenido!\n' +
17     'Las rutas para acceder a los datos son:\n' +
18     '/tasks : CRUD tareas '
19   )
20 })
21
```

Métodos para la interacción final. Hechos en base al ORM Sequelize y los método para el acceso/conexión que facilita el manejo de la base de datos sin necesidad de usar SQL puro

- Método para crear/agregar/guardar una nueva tarea

en el método recibe la información enviada mediante la api, para ser guardada.

el método verifica que la información sea correcta, en case de no serlo muestra en mensaje de error.

en caso de que todo este correcto, guarda la tarea en la base de datos, cumpliendo todos los requisitos de las validaciones para funcionar

```
1 /**
2  * Metodo para crear una nueva tarea
3  * @param {*} req - Objeto de solicitud -> contiene los datos de la nueva tarea
4  * @param {*} res - Objeto de respuesta -> Respuesta con la tarea creada o un error
5  * @returns
6  */
7 export const crearTarea = async (req, res) => {
8   try {
9     // Validar los datos de entrada, verificando si todo es correcto o existe un error
10    const { error } = tareaSchema.validate(req.body);
11    if (error) {
12      return res.status(400).json({ error: error.details[0].message });
13    }
14    // Crear la nueva tarea en la base de datos
15    const nuevaTarea = await Tareas.create(req.body);
16    if (!nuevaTarea) {
17      return res.status(500).json({ error: 'Error al crear tarea' });
18    }
19    // Enviar la tarea creada como respuesta
20    res.status(201).json(nuevaTarea);
21  } catch (error) {
22    console.error('Error al crear tarea:', error);
23    res.status(500).json({ error: 'Error al crear tarea' });
24  }
25 }
```

- Método para obtener todos los registros en la base de datos.  
es decir el método permite consultas todas las tareas existentes en la tabla en la base de datos.  
en caso de no encontrar nada, genera un mensaje de aviso al usuario  
si ocurre un error también genera un mensaje para avisar al usuario

```
1  /**
2   * Metodo para obtener todas Las tareas
3   * @param {*} req - Objeto de solicitud -> retorna La lista de tareas
4   * @param {*} res - Objeto de respuesta -> Respuesta con La lista de tareas o un error
5   * @returns
6   */
7  export const obtenerTareas = async (req, res) => {
8    try {
9      // Obtener todas Las tareas de La base de datos
10     const tareas = await Tareas.findAll();
11     if (!tareas || tareas.length === 0) {
12       return res.status(404).json({ message: 'No se encontraron tareas' });
13     }
14     // Enviar La lista de tareas como respuesta
15     res.json(tareas);
16   } catch (error) {
17     console.error('Error al obtener tareas:', error);
18     res.status(500).json({ error: 'Error al obtener tareas' });
19   }
20  };
```

- Método para obtener un registro de la base de datos según el ID “identificador único”.  
es decir, consulta una sola tarea entregando como parámetro el ID, y recibe como respuesta la información de la tarea en caso de existe, si no encuentra la tarea o hay un error, muestra un mensaje de aviso al usuario

```
1  /**
2   * Metodo para obtener una tarea por ID
3   * @param {*} req - Objeto de solicitud
4   * @param {*} res - Objeto de respuesta -> Respuesta con La tarea encontrada o un error
5   * @returns
6   */
7  export const obtenerTareaPorId = async (req, res) => {
8    try {
9      // Obtener La tarea por ID
10     const tarea = await Tareas.findByPk(req.params.id);
11     if (!tarea) {
12       return res.status(404).json({ error: 'Tarea no encontrada' });
13     }
14     // Enviar La tarea encontrada como respuesta
15     res.json(tarea);
16   } catch (error) {
17     console.error('Error al obtener tarea por ID:', error);
18     res.status(500).json({ error: 'Error al obtener tarea por ID' });
19   }
20  };
```

- Método para actualizar un tarea.

el método permite modificar la información de una tarea específica pasando como primer parámetro el id de la tarea para buscar, en caso de encontrarla usa el segundo parámetro que es la información modificada

```
1  /**
2   * Metodo para actualizar una tarea
3   * @param {*} req - Objeto de solicitud -> contiene los datos de la tarea a actualizar
4   * @param {*} res - Objeto de respuesta -> Respuesta con la tarea actualizada o un error
5   * @returns
6   */
7  export const actualizarTarea = async (req, res) => {
8    try {
9      // Validar los datos de entrada, verificando si todo es correcto o existe un error
10     const { error } = tareaSchema.validate(req.body);
11     if (error) {
12       return res.status(400).json({ error: error.details[0].message });
13     }
14     // Buscar la tarea por ID
15     const tarea = await Tareas.findByPk(req.params.id);
16     if (!tarea) {
17       return res.status(404).json({ error: 'Tarea no encontrada' });
18     }
19     // Actualizar la tarea con los nuevos datos
20     await tarea.update(req.body);
21     res.json(tarea);
22   } catch (error) {
23     console.error('Error al actualizar tarea:', error);
24     res.status(500).json({ error: 'Error al actualizar tarea' });
25   }
26 }
```

- Método para eliminar una tarea, se le entrega como parámetro el ID de la tarea, en caso de que exista y la encuentre la elimina de la base de datos

```
1  /**
2   * Metodo para eliminar una tarea
3   * @param {*} req - Objeto de solicitud -> contiene el ID de la tarea a eliminar
4   * @param {*} res - Objeto de respuesta -> Respuesta con la tarea eliminada o un error
5   * @returns
6   */
7  export const eliminarTarea = async (req, res) => {
8    try {
9      // Buscar la tarea por ID
10     const tarea = await Tareas.findByPk(req.params.id);
11     if (!tarea) {
12       return res.status(404).json({ message: 'Tarea no encontrada' });
13     }
14     // Eliminar la tarea
15     await tarea.destroy();
16     // envia mensaje de "confirmacion"
17     res.status(204).send({
18       message: 'Tarea eliminada exitosamente'
19     });
20   } catch (error) {
21     console.error('Error al eliminar tarea:', error);
22     res.status(500).json({ error: 'Error al eliminar tarea' });
23   }
24 }
25
```

## Especificaciones en el FronEnd

solo se mostrara las partes importantes e interacción con el backend/api

- Metodo para la conexión de la api mediante el servicio conexión al backend mediante una api:

```
1  /**
2   * Configuración de La API
3   * Llamado al backend
4   */
5  const api = axios.create({
6    baseUrl: "http://localhost:3000/api/", // URL de tu backend
7  });
```

- createTarea: permite la conexión con el backend, mandando la informacion para crear una nueva tarea

```
1  /**
2   * Crea una nueva tarea
3   * @param {Tarea} data Datos de La tarea a crear
4   * @returns {Promise<Tarea>} Tarea creada
5   */
6  export const createTarea = async (data) => {
7    const res = await api.post("/tasks", data);
8    return res.data;
9  };
```



- getTareas: obtiene todas las tareas registradas en la base de datos mediante la api, sin parametros

```
1  /**
2   * Obtiene la Lista de tareas
3   * @returns {Promise<Tarea[]>} Lista de tareas
4   */
5  export const getTareas = async () => {
6      const res = await api.get("/tasks");
7      console.log('DATOS ', res.data);
8      return res.data;
9  };
```

- getTareaById: obtiene una tarea de la base de datos, con el parámetro ID de la tarea a buscar

```
1  /**
2   * Obtiene una tarea por su ID
3   * @param {number} id ID de la tarea a obtener
4   * @returns {Promise<Tarea>} Tarea encontrada
5   */
6  export const getTareaById = async (id) => {
7      const res = await api.get(`/tasks/${id}`);
8      return res.data;
9  };
```

- updateTarea: maneja el envío de información para actualizar una tarea, maneja dos parámetros, el ID para buscar/identificar la tarea a actualizar, y los datos nuevos

```
1  /**
2   * Actualiza una tarea existente
3   * @param {number} id ID de la tarea a actualizar
4   * @param {Tarea} data Datos de la tarea a actualizar
5   * @returns {Promise<Tarea>} Tarea actualizada
6   */
7  export const updateTarea = async (id, data) => {
8      const res = await api.put(`/tasks/${id}`, data);
9      return res.data;
10 };
```

- deleteTarea: manejo de la eliminación de una tarea mediante la conexión con la api, con el parámetro ID para obtener la tarea específica a eliminar

```
1  /**
2   * Elimina una tarea por su ID
3   * @param {number} id ID de la tarea a eliminar
4   * @returns {Promise<void>}
5   */
6  export const deleteTarea = async (id) => {
7    const res = await api.delete(`/tasks/${id}`);
8    return res.data;
9  };
```

- Método principal en el Frontend que inicia toda la aplicación donde se establece que se va a manejar por rutas y vistas mediante “navegación” y se llama al método/vista inicial de la aplicación

```
1  createRoot(document.getElementById('root')).render(
2    <StrictMode>
3      <BrowserRouter>
4        <App />
5      </BrowserRouter>
6    </StrictMode>,
7  )
8
```

- Método del componente principal, donde se establecen las rutas y la navegación en la aplicación para la interacción con la api y uso de la información de la base de datos mediante la api y su conexión con el backend

```
1  /**
2   * Componente principal de la aplicación
3   * manejo de las rutas de las vistas/componentes
4   */
5  function App() {
6    return (
7      <Routes>
8        <Route path="/tasks" element={<TaskList />} />
9        <Route path="/task/new" element={<TaskForm />} />
10       <Route path="/task/:id" element={<TaskForm />} />
11       <Route path="/" element={<TaskList />} />
12       <Route path="/task" element={<TaskList />} />
13     </Routes>
14   );
15 }
16
17 export default App;
```

- Tipado de la tabla en el frontend para manejar los campos de la base de datos en vace a un tipo ya existente y no caer en la mala práctica de usar un "any"

```
1  /**
2   * Tipo que representa una tarea
3   */
4  type Tarea = {
5    id: number;
6    title: string;
7    description?: string;
8    status?: string;
9    due_date?: string;
10 };
```

- Método “principal” donde se listan las tareas, y se crea la lógica para la interacción en las tablas se inicia el método, “TaskList”
- se inicializan las variables globales que se van a usar en la vista, ya sea para el almacenamiento de la información o la navegación
- se establece el método para hacer el “cargue” de las tareas, mediante el llamado a la api, al método “obtenerTareas” siendo un método llamado por defecto sin intervención

```
1 function TaskList() {
2   const [tareas, setTareas] = useState<Tarea[]>([]);
3   const navigate = useNavigate();
4
5   /**
6    * Carga las tareas desde la API
7    */
8   const cargarTareas = () => {
9     getTareas()
10      .then((data: SetStateAction<Tarea[]>) => setTareas(data))
11      .catch((error: any) => console.error("Error al obtener tareas:", error));
12   };
13 }
```

- método para eliminar una tarea, “escuchando” si existen un cambio o interacción con el botón que llama la funcionalidad para eliminar, el método también posee una alerta de confirmación que el usuario puede ver, en la cual se le pregunta si realmente quiere eliminar la tarea, o si fue que cambio de opinión o por siguiente, incluso fue un error

```
1  /**
2   * Maneja la eliminación de una tarea
3   * @param id ID de la tarea a eliminar
4   * maneja la alerta de confirmación antes de eliminar la tarea
5   */
6  const handleDelete = async (id: number) => {
7    Swal.fire({
8      title: "¿Estás seguro?",
9      text: "No podrás revertir esta acción",
10     icon: "warning",
11     showCancelButton: true,
12     confirmButtonColor: "#e74c3c",
13     cancelButtonColor: "#95a5a6",
14     confirmButtonText: "Sí, eliminar",
15     cancelButtonText: "Cancelar"
16   }).then(async (result) => {
17     if (result.isConfirmed) {
18       try {
19         await deleteTarea(id);
20         cargarTareas();
21         Swal.fire({
22           title: "Eliminada",
23           text: "La tarea ha sido eliminada.",
24           icon: "success",
25           timer: 1500,
26           showConfirmButton: false
27         });
28       } catch (error) {
29         Swal.fire("Error", "No se pudo eliminar la tarea", "error");
30       }
31     }
32   });
33   };
```

- ahora se ve el return, lo que hace es el renderizado de la vista, es el apartado donde se maneja la interacción del HTML, CSS y TypeScript (lógica) para poder ver e interactuar con la información de las tareas

```
1  /**
2   * Renderiza la lista de tareas
3   * dependiendo de si hay tareas disponibles
4   */
5  return (
6    <div className="container">
7      <div className="header">
8        <h1 className="main-title">Lista de Tareas</h1>
9        <button className="btn-add" onClick={() => navigate("/task/new")}>
10         Agregar Tarea
11       </button>
12     </div>
13
14     <div className="table-wrapper">
15       {tareas.length > 0 ? (
16         <table className="task-table">
17           <thead>
18             <tr>
19               <th>ID</th>
20               <th>Título</th>
21               <th>Descripción</th>
22               <th>Estado</th>
23               <th>Fecha límite</th>
24               <th>Acciones</th>
25             </tr>
26           </thead>
27           <tbody>
28             {tareas.map(tarea => (
29               <tr key={tarea.id}>
30                 <td>{tarea.id}</td>
31                 <td>{tarea.title}</td>
32                 <td>{tarea.description || '-'}</td>
33                 <td>
34                   {tarea.status ? (
35                     <span className={`task-status status-${tarea.status.toLowerCase()}`}>{tarea.status}</span>
36                   ) : '-'}
37                 </td>
38                 <td>{tarea.due_date ? new Date(tarea.due_date).toLocaleDateString() : '-'}</td>
39                 <td>
40                   <button className="btn-edit" onClick={() => navigate(`/task/${tarea.id}`)}>Editar</button>
41                   <button className="btn-delete" onClick={() => handleDelete(tarea.id)}>Eliminar</button>
42                 </td>
43               </tr>
44             )]}
45           </tbody>
46         </table>
47       ) : (
48         <p className="no-tasks">No hay tareas disponibles</p>
49       )}
50     </div>
51   </div>
52 );
```

método/componente donde se maneja la lógica de interacción con los métodos “crearTarea” o “actualizarTarea” según sea el caso, con una condición que identifica si el usuario va a agregar o actualizar

- en este apartado lo primero fue establecer un tipado para las validación requeridas según la tabla en la base de datos, donde se pide desde un mínimo de caracteres hasta un tipo de datos en específico

```
1  /**
2   * Validación del formulario de tareas
3   */
4  const schema = yup.object().shape({
5    title: yup.string()
6      .min(3, "Mínimo 3 caracteres")
7      .max(100, "Máximo 100 caracteres")
8      .required("Título requerido"),
9    description: yup.string()
10     .min(5, "Mínimo 5 caracteres")
11     .max(500, "Máximo 500 caracteres")
12     .required("Descripción requerida"),
13    status: yup.string()
14     .oneOf(["pendiente", "en_progreso", "completada"], "Estado inválido"),
15    priority: yup.string()
16     .oneOf(["1", "2", "3"], "Prioridad inválida"),
17    due_date: yup.date()
18     .min(new Date(), "La fecha debe ser mayor a hoy")
19     .required("Fecha requerida"),
20  });
```

- lo siguiente es establecer el método principal del componente, que maneja el formulario y la lógica

- se inicializan las variables principal, por la cuales se valida el id, en caso de ser actualización, para saber qué información trae, y para la navegación, al completar la operación requerida por el usuario

- método en el que se inicializa los campos que se va a usar en el formulario, y se le dan valores por defectos para su correcto funcionamiento

```
1  function TaskForm() {
2    const { id } = useParams();
3    const navigate = useNavigate();
4
5    /**
6     * Configuración del formulario
7     */
8    const { register, handleSubmit, setValue, formState: { errors } } = useForm({
9      resolver: yupResolver(schema),
10     defaultValues: {
11       title: "",
12       description: "",
13       status: "pendiente",
14       priority: "3",
15       due_date: new Date()
16     }
17   });
```

- en la siguiente parte se declara un efecto, que “escucha” si está actualizando y recibe los tados de la tarea que ya existen en la base de datos, para asignarlos a los campos del formulario para que el usuario pueda realizar la actualización correctamente

```
1  /**
2   * Carga los datos de la tarea si se está editando
3   */
4  useEffect(() => {
5    if (id) {
6      getTareaById(id).then((data: { title: string; description: any; status: any; priority: any; due_date: { split: (arg0: string) => Date[]; }; }) => {
7        setValue("title", data.title);
8        setValue("description", data.description || "");
9        setValue("status", data.status || "pendiente");
10       setValue("priority", data.priority || "3");
11       setValue("due_date", data.due_date && typeof data.due_date === "string" ? new Date(data.due_date) : new Date());
12     });
13   }
14   }, [id, setValue]);
```

- Método de confirmación para guardar o actualizar, este método permite reflejar los cambios en la base de datos, pero posee una función alerta de confirmación que pregunta al usuario si está seguro, si el usuario lo está, guarda en la base de datos los cambio y lo redirige a la vista principal

```
1  /**
2   * Maneja el envío del formulario
3   * @param formData Datos del formulario
4   * maneja una alerta de confirmación antes de enviar los datos
5   * @returns retorna una promesa que se resuelve al enviar los datos
6   */
7  const onSubmit = async (formData: any) => {
8    Swal.fire({
9      title: id ? "¿Actualizar tarea?" : "¿Guardar nueva tarea?",
10     text: id ? "Se actualizarán los datos de esta tarea." : "Se creará una nueva tarea.",
11     icon: "question",
12     showCancelButton: true,
13     confirmButtonColor: "#3085d6",
14     cancelButtonColor: "#d33",
15     confirmButtonText: id ? "Sí, actualizar" : "Sí, guardar",
16     cancelButtonText: "Cancelar"
17   }).then(async (result) => {
18     if (result.isConfirmed) {
19       try {
20         if (id) {
21           await updateTarea(id, formData);
22           Swal.fire("Actualizado", "La tarea ha sido actualizada correctamente.", "success");
23         } else {
24           await createTarea(formData);
25           Swal.fire("Guardado", "La tarea ha sido creada correctamente.", "success");
26         }
27         navigate("/");
28       } catch (error) {
29         console.error("Error al guardar tarea:", error);
30         Swal.fire("Error", "Ocurrió un error al guardar la tarea.", "error");
31       }
32     }
33   });
34   };
```



- visualización de return, donde se maneja el código HTML, CSS y TypeScript para la interacción del formulario en caso de querer guardar o actualizar
- si va a crear/guardar una nueva tarea, el formulario estará vacío, disponible para recibir la información nueva
- si va a actualizar, el formulario tendrá la información de la tarea existente disponible para la modificación
- maneja validación, por lo que no permitirá guardar o actualizar si no cumple con las validaciones establecidas como requisitos

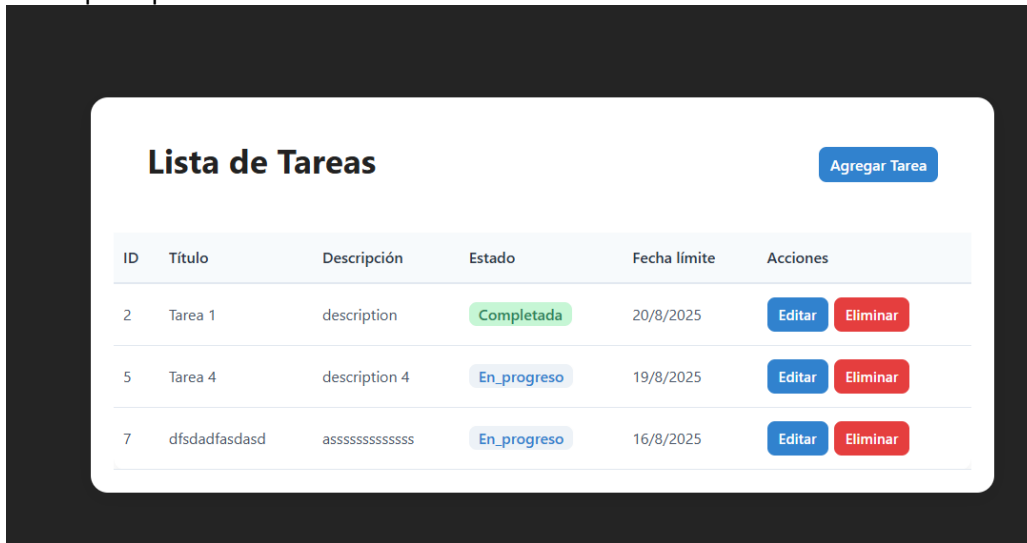
```

1  /**
2   * Renderiza el formulario de tarea
3   * dependiendo de si se está editando o creando una nueva tarea
4   */
5  return (
6    <div className="form-container">
7      <h2 className="form-title">{id ? "Editar Tarea" : "Agregar Tarea"}</h2>
8      <form onSubmit={handleSubmit(onSubmit)}>
9        <label>Título:</label>
10       <input {...register("title")} />
11       {errors.title && <p className="error">{errors.title.message}</p>}
12
13       <label>Descripción:</label>
14       <textarea {...register("description")} />
15       {errors.description && <p className="error">{errors.description.message}</p>}
16
17       <label>Estado:</label>
18       <select {...register("status")}>
19         <option value="pendiente">Pendiente</option>
20         <option value="en_progreso">En Progreso</option>
21         <option value="completada">Completada</option>
22       </select>
23       {errors.status && <p className="error">{errors.status.message}</p>}
24
25       <label>Prioridad:</label>
26       <select {...register("priority")}>
27         <option value="1">Alta</option>
28         <option value="2">Media</option>
29         <option value="3">Baja</option>
30       </select>
31       {errors.priority && <p className="error">{errors.priority.message}</p>}
32
33       <label>Fecha límite:</label>
34       <input type="date" {...register("due_date")} />
35       {errors.due_date && <p className="error">{errors.due_date.message}</p>}
36
37       <button type="submit">{id ? "Actualizar" : "Crear"}</button>
38       <button type="button" onClick={() => navigate("/tasks")}>Cancelar</button>
39     </form>
40   </div>
41 );

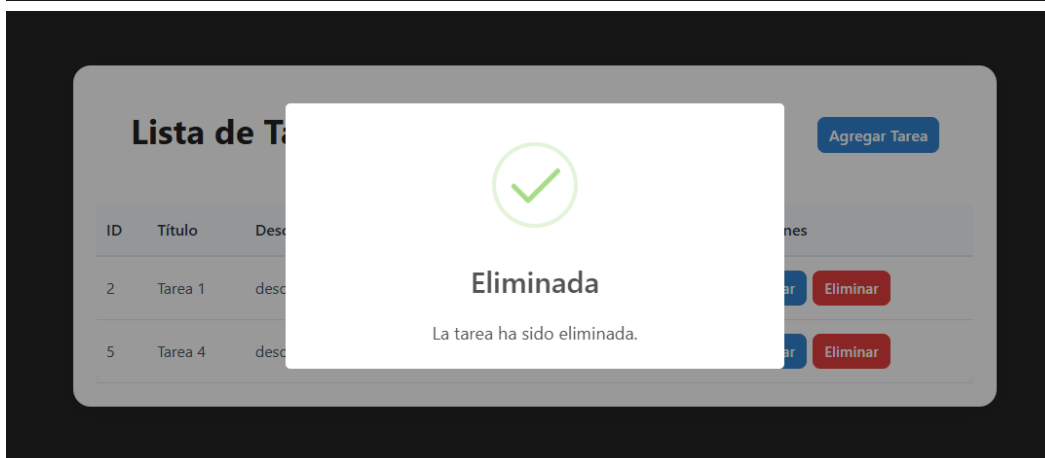
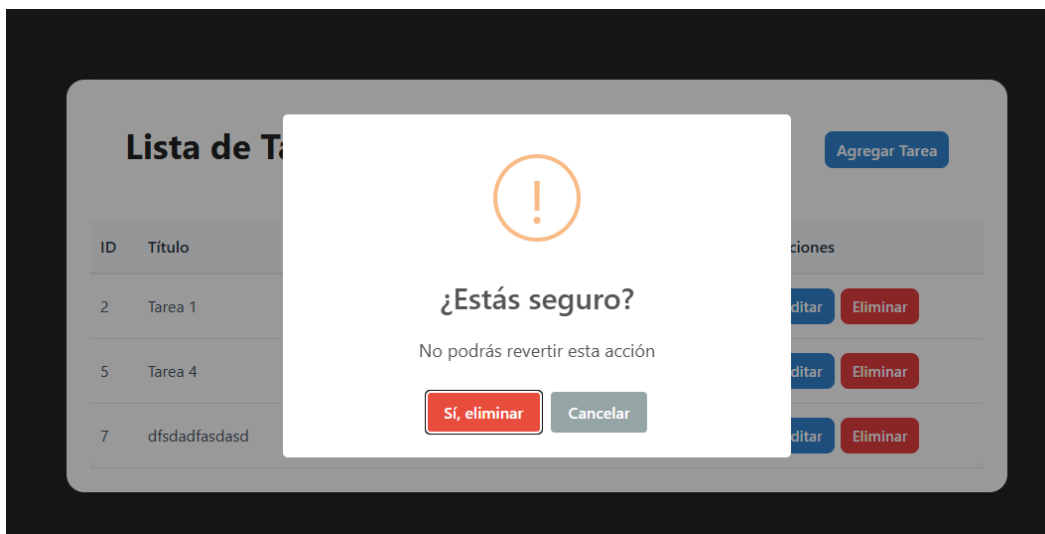
```

- En este apartado, se crean capturas de la aplicación en funcionalidad

- vista principal:



- visualización de la alerta de confirmación al eliminar



- Vista del formulario para agregar una nueva tarea

### Agregar Tarea

Título:

Descripción:

Estado:

Pendiente

Prioridad:

Baja

Fecha límite:

dd/mm/aaaa

Crear Cancelar

- Vista del formulario para actualizar, con la información de la tarea existente y la alerta de confirmación

### Editar Tarea

Título:

Tarea 4

Descripción:

description 4

Estado:

En Progreso

Prioridad:

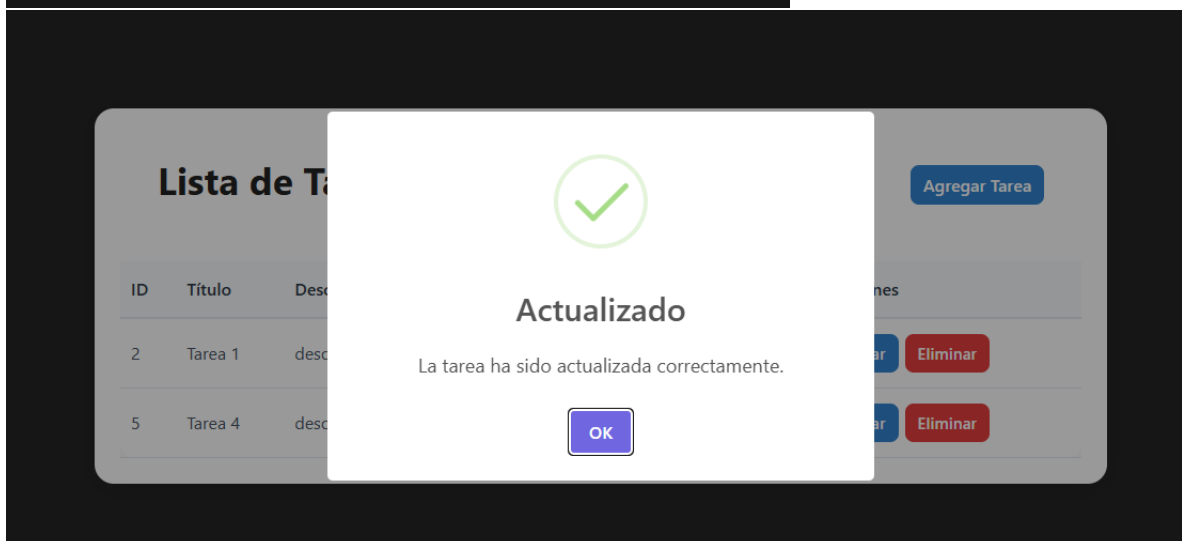
Baja

Fecha límite:

dd/mm/aaaa

Actualizar

Cancelar



Esas son todas las imágenes de confirmación y explicación del código, y de la visualización de cómo trabaja la vista